

Phase 3: Interface Binding · AI Zone: ON — required tool, prompt logs mandatory Session length: ~90 min lecture + handoff to lab **Stack:** Concepts and pseudocode primary; a Blade example included. Applies to Blade, React, Angular, Vue.

1. Explain what the view layer does — render state as user interface.
2. Break a screen into reusable **components**.
3. Explain how data becomes UI: lists, conditionals, dynamic values.
4. Translate a Week 2 wireframe into a component structure and markup.
5. Use AI as a required tool — and keep an honest prompt log.

Time	Segment
0:00-0:10	Framing: Phase 3 — AI turns ON
0:10-0:26	Part A — What the view layer does
0:26-0:44	Part B — Components: breaking UI into pieces
0:44-0:47	Discussion break 1
0:47-1:04	Part C — Rendering state (lists & conditionals)
1:04-1:14	Part D — From wireframe to markup
1:14-1:16	Discussion break 2
1:16-1:26	Part E — Using AI well (and logging it)
1:26-1:30	Handoff to lab

- **AI is required**, not optional — this is how professionals work.
- **Prompt logs are mandatory**. Every deliverable from here carries a log of the key prompts you used and which code is AI-generated, AI-modified, or hand-written.
- **You still own the code**. AI drafts; you review, fix, and defend it. Using AI is not cheating — hiding that you used it is.

Frame it honestly: AI will let you move faster this phase. The skill now is *directing and reviewing* it, not typing every line.

```
[ logic layer ] --- data ---> [ view layer ] --- renders ---> [ what the user sees ]
    Phase 2                Phase 3
```

Tie it back: in Week 3 you chose monolithic or decoupled. That choice decides *where* rendering happens (server vs client), but the view still renders state either way.

Part B — Components: Breaking UI Into Pieces (18 min)

A **component** is a self-contained, reusable chunk of UI — a card, a form, a nav bar, a single list row. Instead of one giant HTML file, you compose the screen from pieces.

Why components:

- **Reuse** — build an "order row" once, use it for every order.
- **Isolation** — a bug in the card is contained to the card.
- **Readability** — a page reads as a list of named pieces, not 400 lines of markup.

Composition — components nest. A page contains a list; the list contains rows; a row contains a status badge. Draw this tree on the board.

Inputs (props/data) — a component takes data and renders it. The same "order row" component renders differently for order #42 vs #43 because it receives different data. This is the reuse mechanism.

This maps to your framework: Blade components/partials, React/Vue/Angular components — same concept, different syntax.

Discussion break 1: "Look at your orders list wireframe. What's the one component you'd build once and repeat?" (The row.)

Part C — Rendering State: Lists & Conditionals (17 min)

Two patterns turn data into UI. Almost every screen is built from these.

Lists — map data to markup. You don't hand-write ten order rows; you loop over the data and render one row per item.

```
for each order in orders:
  render OrderRow(order)
```

Conditionals — show different UI for different state. This is where the wireframe states from Week 2 become real:

```
if orders is empty:    render EmptyState()
else if loading:       render Spinner()
else if error:         render ErrorMessage(error)
else:                 render the list
```

Dynamic values — slot data into markup (the order's item name, its quantity, its status). The static wireframe had placeholder boxes; now real values fill them.

Emphasize: the empty / loading / error states you wireframed in Week 2 are not optional polish — they're conditionals you write now. A list with no empty state shows a blank screen to every new user.

Part D — From Wireframe to Markup (10 min)

Walk the translation from a Week 2 wireframe to real structure:

1. **Identify the components** — the wireframe's boxes become named pieces (header, list, row, form).
2. **Use semantic HTML** — headings, lists, forms, buttons. Structure first.
3. **Style lightly** — layout and spacing to match the wireframe. Keep it clean, not fancy; polish comes later.
4. **Wire in data** — replace placeholder labels with dynamic values and the list/conditional patterns.

The wireframe is the contract; the markup realizes it. If a screen or state was in the wireframe, it should exist now.

Discussion break 2: "Your wireframe showed an empty state. Where in the markup does that live?" (A conditional — rendered when the list is empty.)

Part E — Using AI Well (and Logging It) (10 min)

This is the first week AI is a required tool. Model *good* use.

Good prompt patterns to demonstrate:

- "Scaffold an accessible HTML/CSS component for an order row showing item, quantity, and a status badge."
- "Given this wireframe description, generate the empty-state markup for an orders list."
- "Review this view component for accessibility and semantic HTML issues."

The discipline: AI scaffolds fast, but it produces confident, plausible markup that can be subtly wrong — inaccessible, non-semantic, or not matching your wireframe. **You review and fix it.** This is a preview of Week 9, where reviewing AI code becomes

the headline skill.

The prompt log (mandatory now): in `/docs/ai-notes/`, record the key prompts used and mark which components are AI-generated, AI-modified, or hand-written. Undisclosed AI use is an integrity violation.

In Laravel (one concrete example)

Blade components express the component idea directly. A reusable order row:

```
{{-- resources/views/components/order-row.blade.php --}}
<tr>
    <td>{{ $order->item }}</td>
    <td>{{ $order->qty }}</td>
    <td><span class="badge">{{ $order->status }}</span></td>
</tr>
```

```
{{-- the list: loop + empty-state conditional --}}
@forelse ($orders as $order)
    <x-order-row :order="$order" />
@empty
    <p>No orders yet.</p>
@endforelse
```

`@forelse/@empty` is the list-plus-empty-state pattern built in. The concept — components, lists, conditionals — transfers to React/Vue/Angular.

Handoff to Lab (4 min)

Teams translate their Week 2 wireframes into real HTML/CSS views: identify components, build each screen, and render the empty/loading/error states. AI is on and encouraged for scaffolding — but every team keeps a prompt log, and reviews what AI produces. Point them to the Week 6 Lab Handout.

Instructor Notes / Common Pitfalls

- **AI-on doesn't mean AI-does-everything.** Watch for teams pasting AI markup without reading it. Require them to explain any component they submit.
- **The forgotten states return.** Teams will build the happy-path list and skip empty/error again. Hold them to the wireframe.
- **Component sprawl vs. one giant file.** Nudge toward a few sensible components, not 40 tiny ones and not one 400-line page.
- **Keep styling in check.** This week is structure and state, not a design competition. Polish is Week 8+.
- **Prompt logs start now.** Make the first one a habit even if small — it's graded in Deliverable 3.